

Assignment 1

Course: Deep Learning

Instructor: Dr. Ali Irtaza

Department of Electrical Engineering

Institute of Space Technology

Submitted by:

Zawar Khan

September 23, 2026

Q1. Function approximation with a deep neural network

Use deep neural networks to estimate the functions (a) $f(x) = x^2$, (b) $f(x) = \sin(x)$, and (c) $f(x) = |x|^2$. Try data sets of different sizes, e.g. 10, 25, 50, 100 points for each function. Use 80% of the data for training and 20% for testing.

(a) Plot the test and training error for different sizes of the data sets.

(b) Try reversing the sign of the weights and observe the performance.

Hint: Consider starting with a two-layer network with 3 hidden units with tanh activation and a linear output unit.

Forward equations and training objective

The hidden units apply the hyperbolic tangent:

$$h_1 = \tanh(w_1x + b_1), \quad (1)$$

$$h_2 = \tanh(w_2x + b_2), \quad (2)$$

$$h_3 = \tanh(w_3x + b_3). \quad (3)$$

The output unit is **linear**: a weighted sum of the hidden activations plus a bias,

$$\hat{y} = W_1h_1 + W_2h_2 + W_3h_3 + b_{\text{out}}. \quad (4)$$

The training is done by minimising the **mean squared error** (MSE)

$$\text{MSE} = \frac{1}{N} \sum_{n=1}^N (\hat{y}_n - y_n)^2 = \frac{1}{N} \sum_{n=1}^N (\hat{y}_n - f(x_n))^2. \quad (5)$$

Reaching exactly zero MSE is not expected in practice; a small positive value is an acceptable stopping region (limits to be set later). Moreover, Weights are initialised randomly: hidden (w_1, b_1) , (w_2, b_2) , (w_3, b_3) and output $(W_1, W_2, W_3, b_{\text{out}})$.

Network architecture (block diagram)

Figure 1 shows one scalar input x (not counted as a layer), one hidden layer with three tanh neurons, and one linear output $\hat{y}$.

(a) Target $f(x) = x^2$

Training uses a **fixed epoch budget** (not $\text{MSE} \rightarrow 0$). Driving the loss all the way to zero would increase time and computational cost sharply, so we keep a finite schedule and

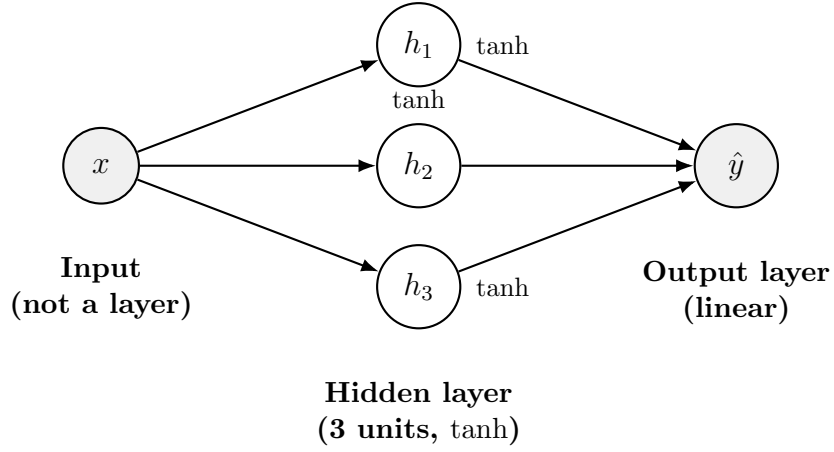


Figure 1: Initial network for Q1: one input x , one hidden layer with three tanh units, and one linear output $\hat{y} = W_1 h_1 + W_2 h_2 + W_3 h_3 + b_{\text{out}}$.

report both error and time versus N . Datasets of size $N \in \{10, 25, 50, 100\}$ are drawn on $x \in [-2, 2]$, with 80% training and 20% testing. Full source code is available in the course repository [2].

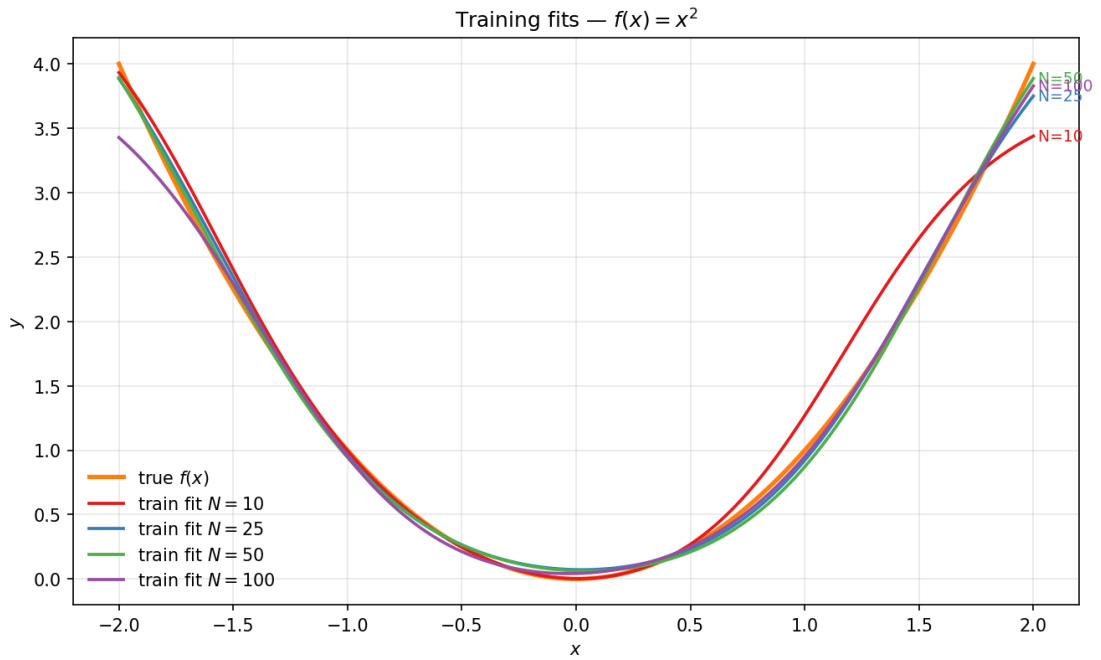


Figure 2: Training view for $f(x) = x^2$: orange solid curve is the true $f(x)$; coloured curves are network fits for each training-set size N (same colour coding as the test plot).

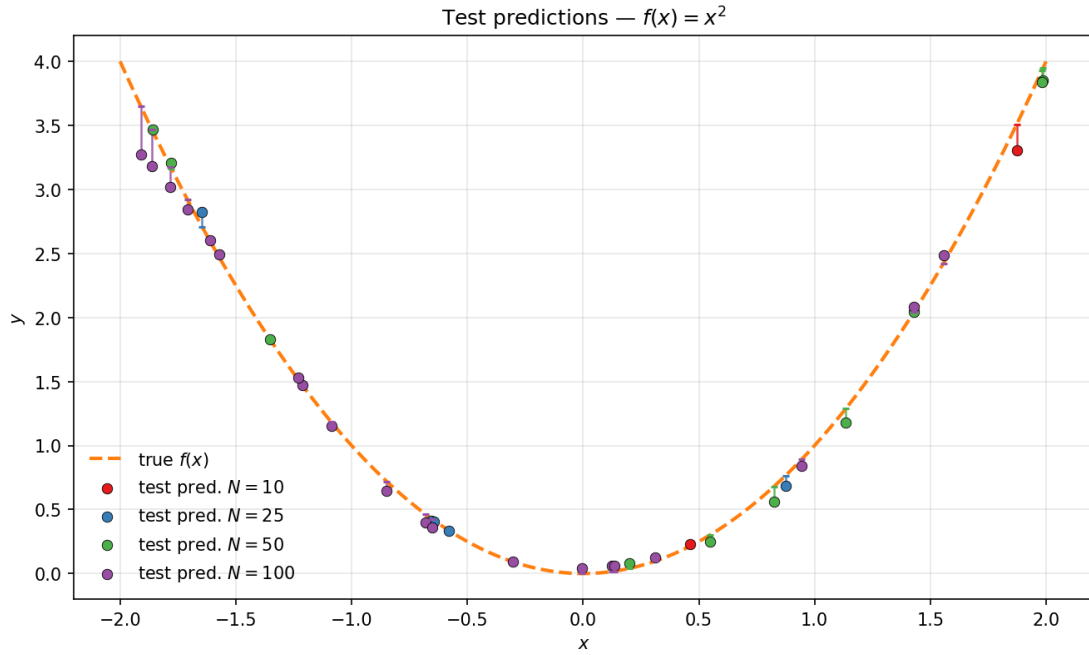


Figure 3: Test view for $f(x) = x^2$: orange dotted curve is the true $f(x)$; coloured markers are network predictions on the held-out 20% test samples. Vertical bars (same colour per N) run from the true value to the prediction (residual / Excel-style error bars).

Python implementation (modular core)

```

1 import numpy as np
2
3 def forward(x, w, b, W, b_out):
4     h = np.tanh(x[:, None] * w[None, :] + b[None, :])
5     return h @ W + b_out
6
7 def mse(y_hat, y_true):
8     return float(np.mean((y_hat - y_true) ** 2))
9
10 # Fixed epochs (not until MSE = 0).

```

Listing 1: Forward pass, MSE, and training entry point

Conclusion plots (MSE and time cost)

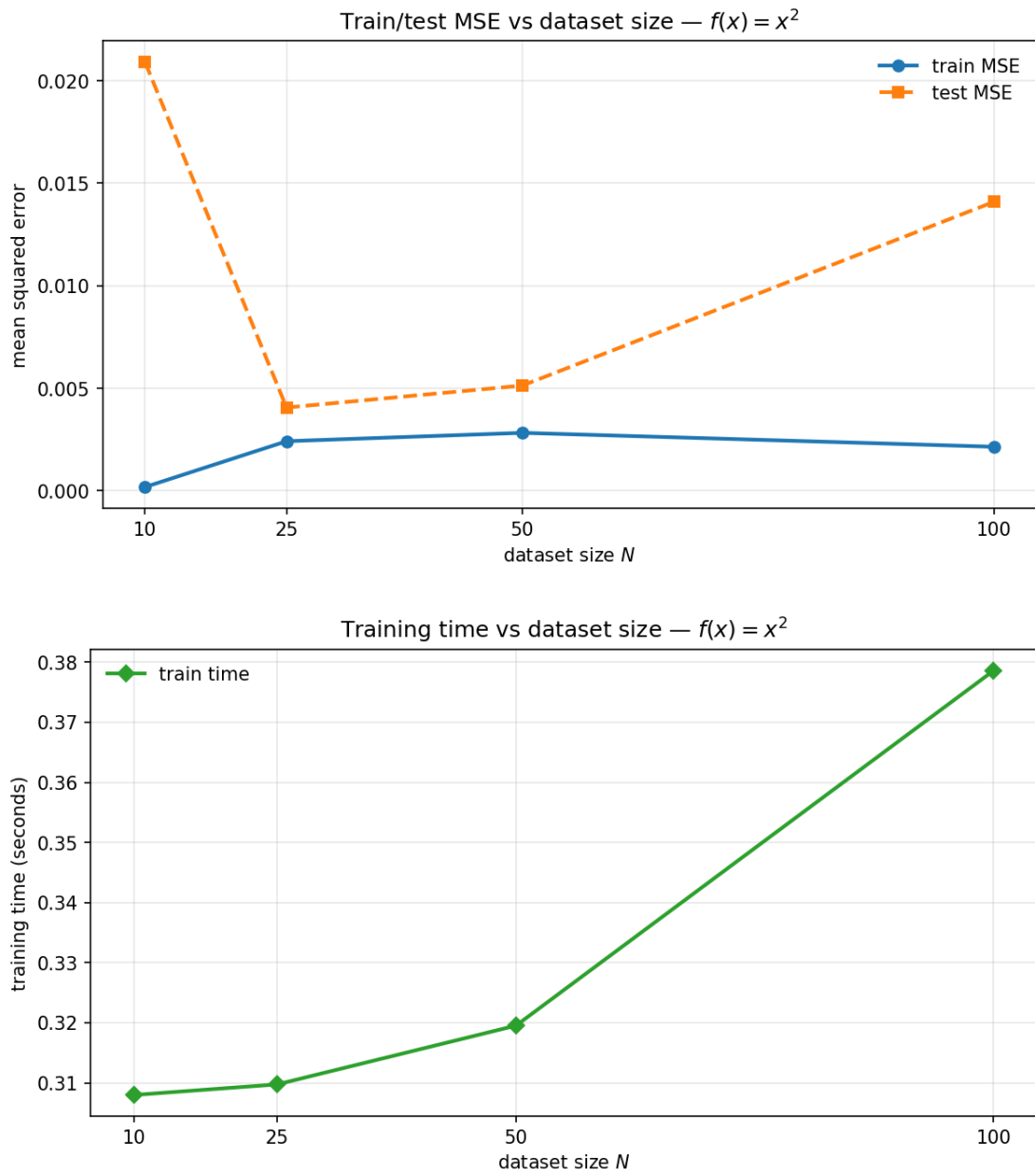


Figure 4: Train/test MSE (top) and training wall-clock time (bottom) versus dataset size for $f(x) = x^2$ under a fixed epoch budget.

Final learned weights

Parameter	$N = 10$	$N = 25$	$N = 50$	$N = 100$
$w1$	1.4307	-0.0079	0.0064	1.1487
$b1$	-1.6728	1.0832	-1.4814	-1.8338
$w2$	1.3465	1.2673	1.1195	0.0442
$b2$	0.9632	-1.9263	1.8504	1.5016
$w3$	-1.5196	-1.1888	-1.2290	-1.3283
$b3$	-2.4299	-1.8812	1.9644	-1.8412
$W1$	2.0422	1.3332	-2.2249	2.7916
$W2$	-0.5152	2.5378	-2.9753	2.1109
$W3$	2.0939	2.7647	-2.7992	2.1923
b_{out}	4.3518	4.0848	3.5800	2.8687

Table 1: Final trained parameters for $f(x) = x^2$ at each dataset size N .

Analysis

The training plot shows that using a larger dataset generally improves how closely the network follows the true $f(x) = x^2$ curve. However, when the dataset size is increased to $N = 100$, the fitted curves start to deviate and show overfitting behaviour. The MSE plot shows that, overall, the error trend improves as more training data are used, while the test view indicates that the $N = 25$ set performed best for this target; further increasing the dataset size raised the observed test error rate for this run.

Target $f(x) = \sin(x)$

The same network and fixed-epoch MSE training are used, but the input domain for this target is sampled in **degrees** on $x \in [-180^\circ, 180^\circ]$ and converted to radians before evaluating the sine and before feeding the network ($x_{\text{rad}} = \pi x_{\text{deg}}/180$), so plots show a clear period while tanh units are not saturated by degree-scale inputs. Source: [2].

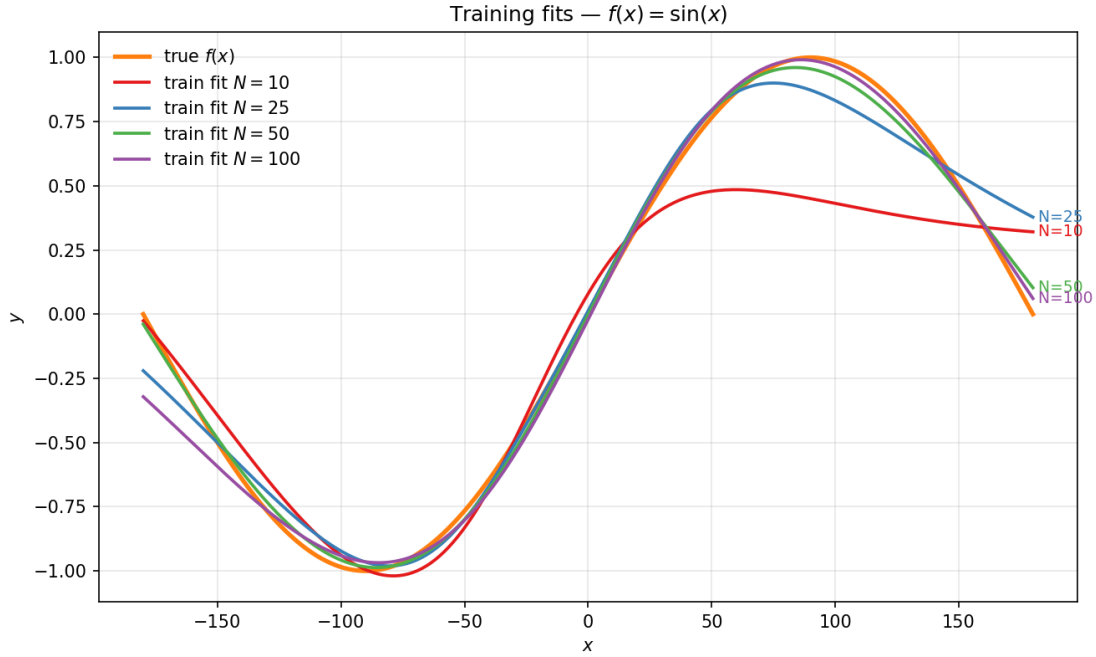


Figure 5: Training fits for $f(x) = \sin(x)$ with x in degrees (converted to radians for the target).

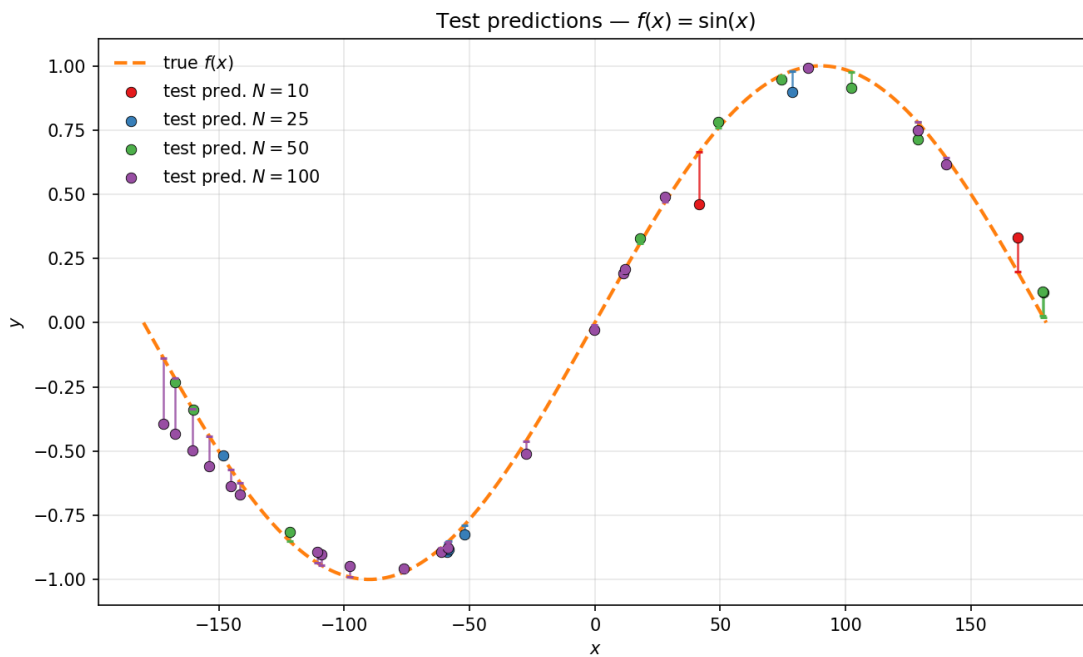
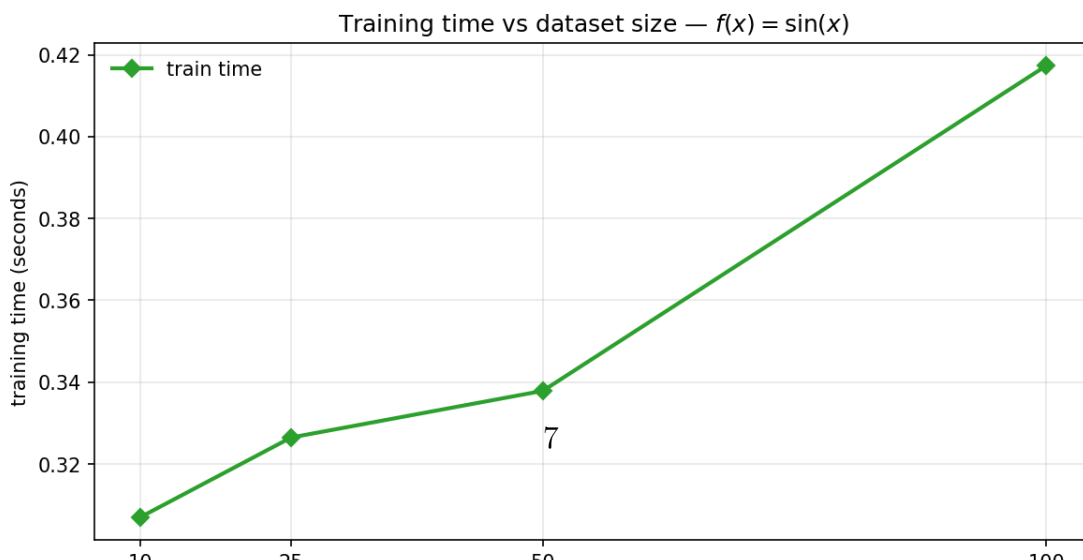
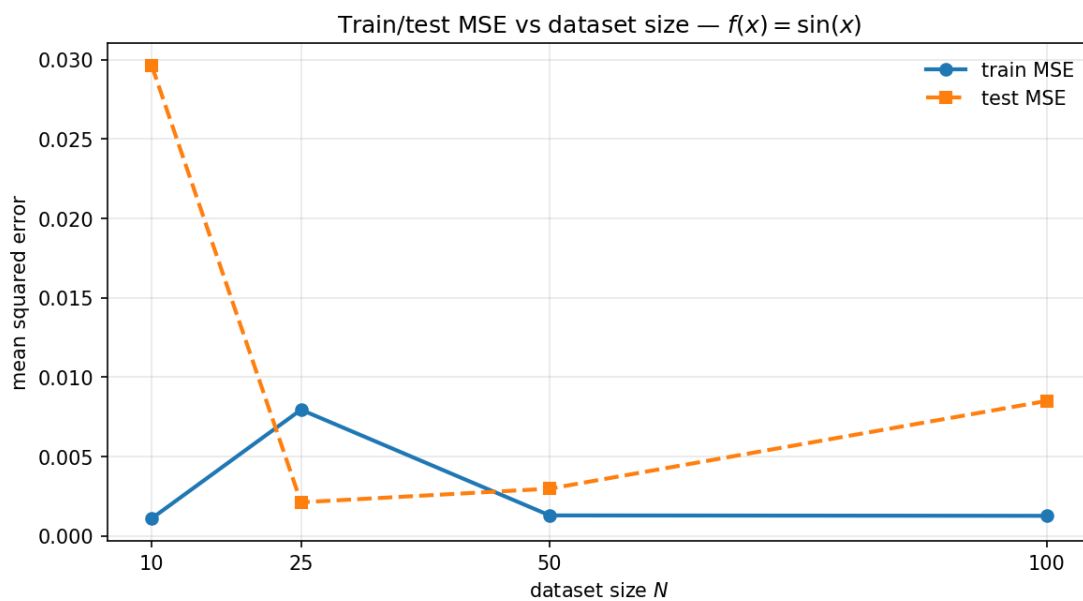


Figure 6: Test predictions with residual bars for $f(x) = \sin(x)$.

Conclusion plots (MSE and time cost)



Final learned weights

Parameter	$N = 10$	$N = 25$	$N = 50$	$N = 100$
$w1$	-0.6496	0.2715	0.5086	0.6894
$b1$	0.6378	0.2113	-1.3173	0.0033
$w2$	1.0591	1.1875	0.5399	-0.6209
$b2$	0.4671	-0.5209	1.5921	1.7443
$w3$	-0.4820	-1.1292	-0.6585	-0.4918
$b3$	-1.1013	-0.7453	0.0216	-1.1461
$W1$	0.2095	-2.8783	-1.6950	2.0665
$W2$	1.5244	1.0871	-1.7889	1.4735
$W3$	1.6642	-1.1662	-2.1990	1.3662
b_{out}	0.6301	0.3950	0.2234	-0.2986

Table 2: Final trained parameters for $f(x) = \sin(x)$ at each dataset size N .

Analysis

Over this interval the training plot shows that for smaller datasets the fitted curves do not match the true sinusoid well. Increasing the dataset size finds better weights, so the learned behaviour becomes finer and closer to the true curve. In particular, $N = 50$ is a good approximation for this target: increasing the data further mainly raises computational and timing cost under our fixed-epoch schedule. The test results also show that for $N = 100$ the error increases and finding the weights takes longer than for $N = 50$, even though a longer budget might eventually do better—so the computation cost must be understood as well. The train-time plot generally rises with N , with some variability at $N = 100$, where accuracy was still worse: with the epoch budget exhausted the optimiser did not fully converge and the large-data run appears to oscillate more. The MSE plot confirms this analysis.

Target $f(x) = |x|^2$

For real x , $|x|^2 = x^2$, so this target produces the same parabola as part (a). The experiment is therefore not repeated.

Q2. Properties of the logistic sigmoid

Show that the logistic sigmoid function satisfies $\sigma(-a) = 1 - \sigma(a)$ and that its inverse is given by $\sigma^{-1}(y) = \log\{y/(1 - y)\}$.

The logistic sigmoid is defined by

$$\sigma(a) = \frac{1}{1 + e^{-a}}. \quad (6)$$

(i) $\sigma(-a) = 1 - \sigma(a)$

Substitute $-a$ into (6):

$$\sigma(-a) = \frac{1}{1 + e^{-(-a)}} = \frac{1}{1 + e^a}. \quad (7)$$

Next,

$$1 - \sigma(a) = 1 - \frac{1}{1 + e^{-a}} = \frac{(1 + e^{-a}) - 1}{1 + e^{-a}} = \frac{e^{-a}}{1 + e^{-a}}. \quad (8)$$

Multiply the numerator and denominator of $\sigma(-a)$ by e^{-a} :

$$\sigma(-a) = \frac{1}{1 + e^a} \cdot \frac{e^{-a}}{e^{-a}} = \frac{e^{-a}}{e^{-a} + 1} = \frac{e^{-a}}{1 + e^{-a}}. \quad (9)$$

Therefore

$$\sigma(-a) = 1 - \sigma(a). \quad (10)$$

(ii) **Inverse** $\sigma^{-1}(y) = \log\{y/(1 - y)\}$

Set $y = \sigma(a)$ and solve for a :

$$y = \frac{1}{1 + e^{-a}}. \quad (11)$$

Multiply both sides by $1 + e^{-a}$:

$$y(1 + e^{-a}) = 1. \quad (12)$$

Expand:

$$y + y e^{-a} = 1. \quad (13)$$

Isolate the exponential:

$$y e^{-a} = 1 - y, \quad (14)$$

$$e^{-a} = \frac{1 - y}{y}. \quad (15)$$

Take the natural logarithm of both sides:

$$-a = \log\left(\frac{1 - y}{y}\right). \quad (16)$$

Using $-\log x = \log(1/x)$,

$$a = \log\left(\frac{y}{1-y}\right). \quad (17)$$

Hence

$$\sigma^{-1}(y) = \log\left\{\frac{y}{1-y}\right\}. \quad (18)$$

Q3. Consider two sets of outputs

Set A: outcomes with *minimum to low* entropy

Set B: outcomes with *medium to high* entropy

(a) Which set is more meaningful for decreasing unpredictable behaviour?

Shannon entropy (analytical)

Shannon's information theory defines the unpredictability of a discrete random variable [1]. The Shannon measure of unpredictability is the **Shannon entropy**. For a single outcome i with probability p_i , Shannon defined the *surprise* (self-information) as

$$\text{information}(i) = \log_b\left(\frac{1}{p_i}\right) = -\log_b(p_i). \quad (19)$$

Interpretation of (19):

- If p_i is close to 1 (a near-certain event), then $\log(1/p_i)$ is close to 0 — almost no information/surprise.
- If p_i is close to 0 (a rare event), then $\log(1/p_i)$ is large — a large amount of information/surprise.

For *many* outcomes, the entropy of the full distribution is the **probability-weighted sum** of these surprises:

$$H(X) = \sum_i p_i \text{information}(i) = \sum_i p_i (-\log_b p_i) = -\sum_i p_i \log_b p_i, \quad (20)$$

where p_i is the probability of outcome i and the base b chooses the unit (bits when $b = 2$). Equation (20) is the general multi-outcome formula used for Set A and Set B below.

Special case (two outcomes). For a Bernoulli system with probabilities p and $1 - p$, (20) reduces to the **binary entropy**

$$H(p) = -p \log_2 p - (1 - p) \log_2 (1 - p). \quad (21)$$

$H(p)$ is *low* when p is near 0 or near 1 (highly predictable outcomes) and reaches its *maximum* at $p = 0.5$ (maximum randomness). Thus “minimum to low entropy” corresponds to probability mass concentrated on nearly certain outcomes, while “medium to high entropy” corresponds to more uniformly uncertain outcomes.

Figures

Figure 8 shows the binary-entropy special case $H(p)$. Entropy is smallest near the extremes of p .

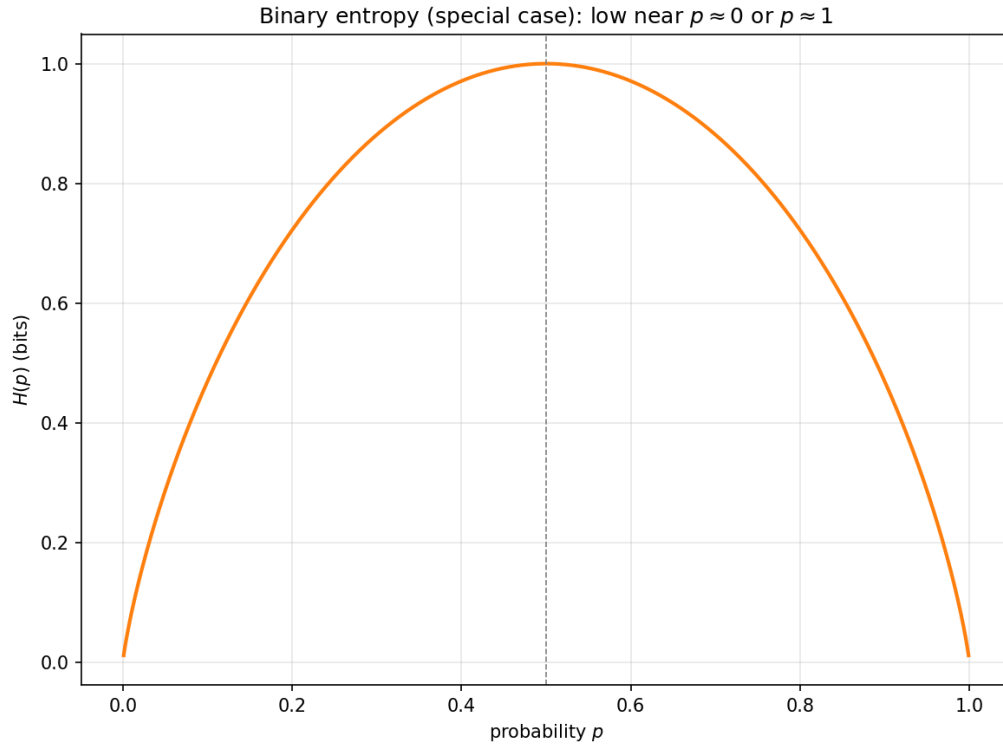


Figure 8: Binary Shannon entropy $H(p)$ versus probability p (special case of two outcomes). Low entropy occurs for very small or very large p .

To connect (20) to Set A and Set B, we form two sample sets whose outcomes are integers from 1 to 10 (size $N = 100$):

- **Set A:** draws are random, but a few numbers appear much more often (peaked frequencies $\Rightarrow$ low entropy).
- **Set B:** draws are nearly uniform, so the same number rarely dominates (spread frequencies $\Rightarrow$ high entropy).

Empirical probabilities are $p_i = n_i/N$. Importantly:

- The **top histograms** show only *counts* (how often each outcome appears). They are *not* an entropy curve.
- **Entropy** is a *single number* for each whole set, $H = -\sum_i p_i \log_2 p_i$, shown in the **bottom panel** of Figure 9.

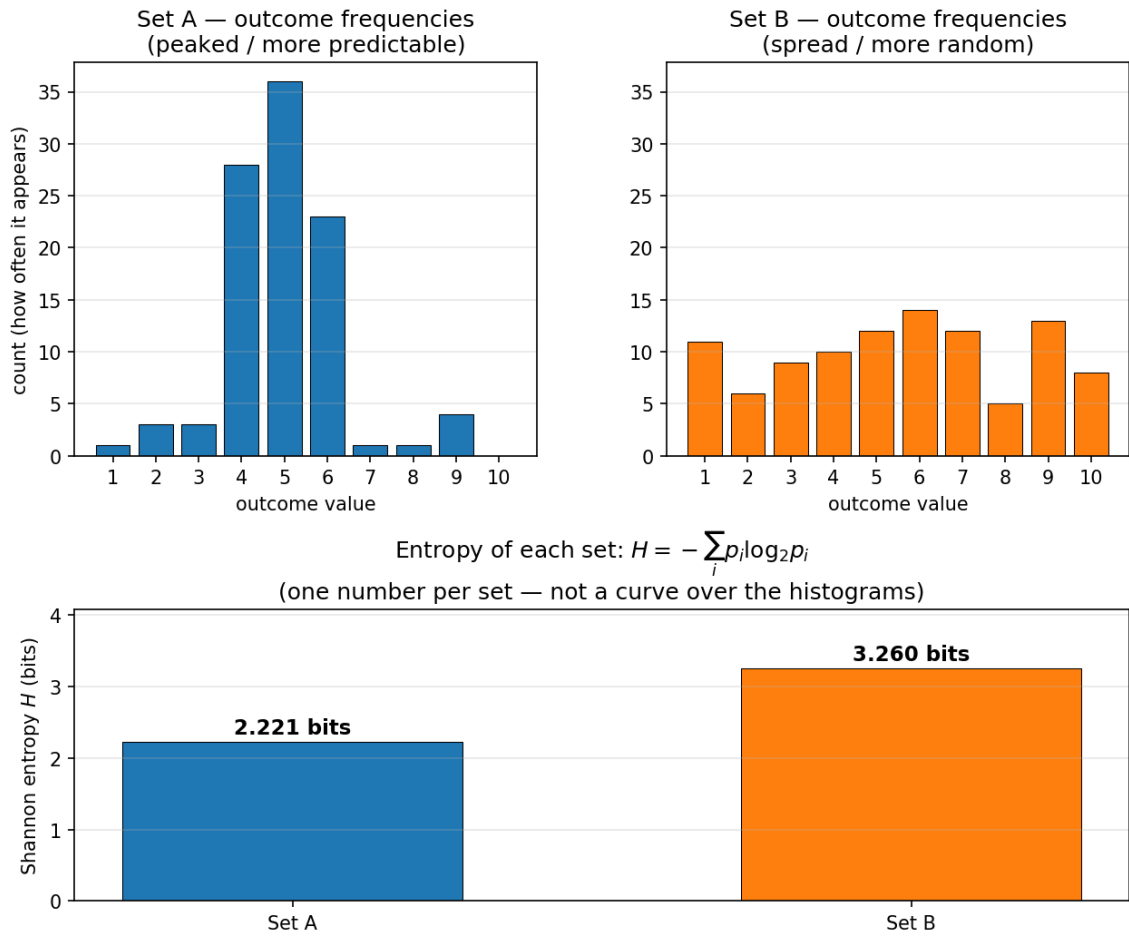


Figure 9: Top: outcome frequencies for Set A and Set B (values 1–10). Bottom: Shannon entropy of each set (one bar per set). The histograms describe the data; the bottom panel is where entropy is compared.

Conclusion

Entropy is a direct measure of randomness / unpredictability. To *reduce* unpredictable behaviour we need outcomes that are as predictable as possible — concentrated probability mass and **low Shannon entropy**. From (19), (20), and Figures 8–9, that regime is exactly **Set A**. Set B corresponds to greater uncertainty and is less suitable if the design goal is to reduce randomness.

Note. Full plotting scripts and libraries are in the course repository [2].

Python Implementation

```
1  import numpy as np
2
3  def shannon_entropy(probs, base=2.0):
4      """H = -sum p_i log_b(p_i) (weighted sum over many
5          outcomes)."""
6      p = np.asarray(probs, dtype=float)
7      p = p / p.sum()
8      nz = p > 0
9      return float(-(p[nz] * (np.log(p[nz]) / np.log(base))).sum
10                     ())
11
12 def binary_entropy(p, base=2.0):
13     """Special case: H(p) for Bernoulli(p); low near p=0 or p
14         =1."""
15     p = np.clip(np.asarray(p, dtype=float), 0.0, 1.0)
16     h = np.zeros_like(p, dtype=float)
17     mid = (p > 0) & (p < 1)
18     pm = p[mid]
19     h[mid] = -(pm * np.log(pm) + (1 - pm) * np.log(1 - pm)) /
20               np.log(base)
21     return h
```

Listing 2: Shannon entropy and binary entropy

Q4. Normal distribution and maximum entropy

If we know the mean μ and standard deviation σ , is it true that the Normal (Gaussian) distribution has the maximum entropy?

Differential entropy

For a continuous random variable with density $p(x)$, Shannon's discrete sum becomes an integral the **differential entropy**

$$H(X) = - \int_{-\infty}^{\infty} p(x) \log p(x) dx \quad (22)$$

This is the continuous analogue of (20). Standard results in information theory show that, among all continuous densities that share a fixed mean μ and fixed variance σ^2 , the Gaussian density uniquely maximises $H(X)$ [3].

Entropy of a Gaussian

For $X \sim \mathcal{N}(\mu, \sigma^2)$ the integral (22) evaluates in closed form to

$$H(X) = \frac{1}{2} \log_2(2\pi e \sigma^2) \quad (\text{bits}). \quad (23)$$

Two observations follow immediately from (23):

- $H(X)$ depends only on the variance σ^2 , *not* on the mean μ . Entropy measures unpredictability (spread), not where the bell curve is centred. Shifting the curve left or right does not change how uncertain the outcome is.
- As $\sigma \rightarrow 0$ (a very narrow spike) the entropy becomes very small: you are almost certain of the value. As σ grows (a very wide curve) the entropy grows: values can land almost anywhere, so uncertainty is high.

Maximum-entropy principle

The entropy under given constraints is maximized by the Normal distribution.

1. $\int p(x) dx = 1$ (a valid density),
2. $\mathbb{E}[X] = \mu$ and $\text{Var}(X) = \sigma^2$ (known mean and spread),

Equivalently: among all continuous distributions that share the same μ and σ^2 , the Gaussian is the one with the **highest** differential entropy. Any other shape with the same mean and variance (skewed, multimodal, truncated, ...) secretly encodes *extra*

structure beyond what μ and σ^2 alone justify, and that extra structure *reduces* entropy. Gaussian is therefore the “least additional-assumption” choice consistent with those two facts the **maximum entropy principle** [3].

Illustration (same μ, σ^2 , different shapes)

Figure 10 fixes $\mu = 0$ and $\sigma = 1$ and plots three densities that all have this mean and variance: Gaussian, Laplace, and Uniform. The left panel shows the shapes; the right panel shows their differential entropies.

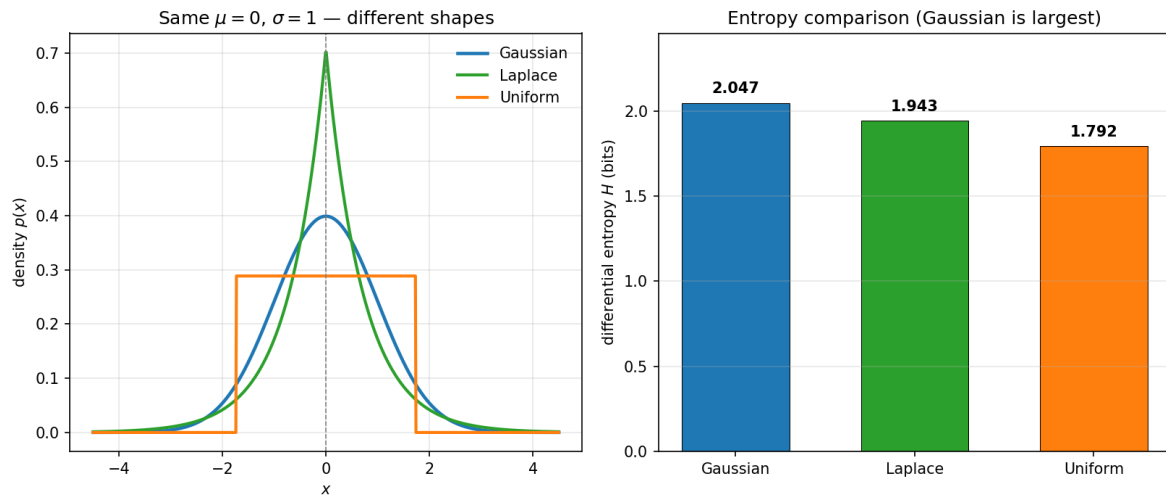


Figure 10: Left: three PDFs with identical μ and σ^2 . Right: differential entropy of each. The Gaussian has the largest H ; the other shapes add extra structure (sharper peak or hard cut-offs) and therefore lower entropy.

Q5. Find and plot the derivatives of (a) ReLU, (b) Sigmoid, and (c) Hyperbolic Tangent functions.

(a) ReLU

Analytical Form

The Rectified Linear Unit (ReLU) is defined as

$$\text{ReLU}(x) = \max(0, x) = \begin{cases} 0, & x < 0, \\ x, & x \geq 0. \end{cases} \quad (24)$$

Differentiating piecewise (taking the subgradient at $x = 0$ to be 0, as is common in practice) yields

$$\text{ReLU}'(x) = \begin{cases} 0, & x \leq 0, \\ 1, & x > 0. \end{cases} \quad (25)$$

Figure 1 shows the ReLU activation function and its derivative.

Plot

The derivative in python code is simple finite check in the limits given while for the derivative at Zero its taken as 0.

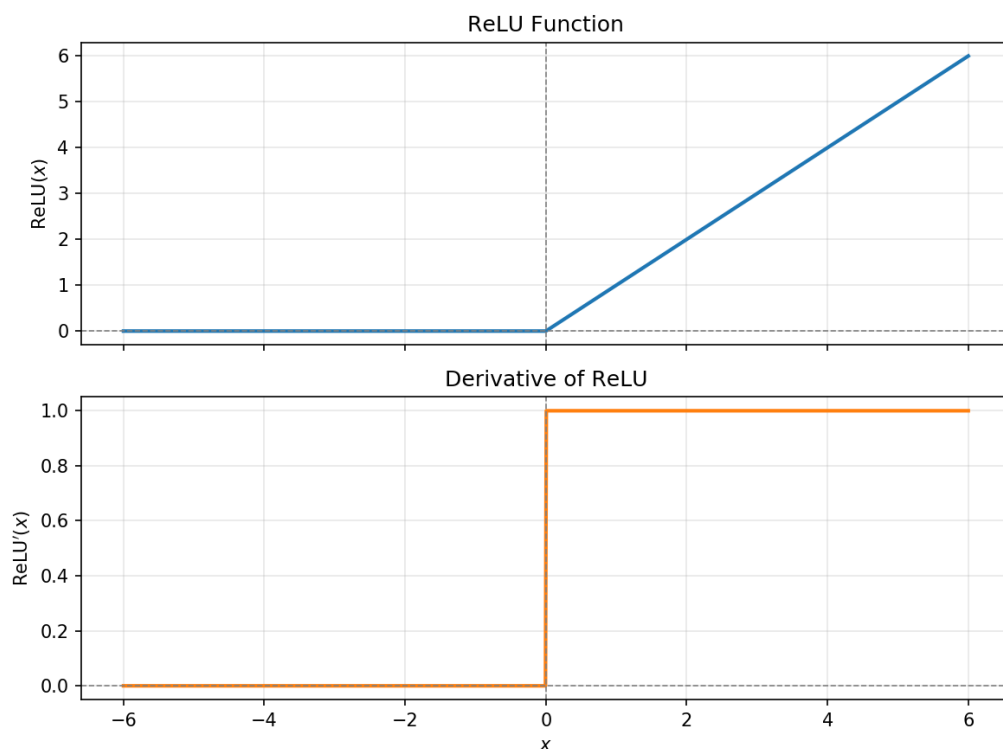


Figure 11: ReLU activation (top) and its derivative (bottom).

Python Implementation

For the computation of derivatives we have used the NumPy library.

```
1 import numpy as np
2 def relu(x):
3     return np.maximum(0.0, x)
4
5 def relu_derivative(x):
6     return (x > 0).astype(float)
```

Listing 3: ReLU and its derivative

(b) Sigmoid

Analytical Form

The logistic sigmoid is

$$\sigma(x) = \frac{1}{1 + e^{-x}}. \quad (26)$$

Differentiating with respect to x :

$$\sigma'(x) = \frac{e^{-x}}{(1 + e^{-x})^2} = \frac{1}{1 + e^{-x}} \cdot \frac{e^{-x}}{1 + e^{-x}} = \sigma(x) (1 - \sigma(x)). \quad (27)$$

Thus

$$\sigma'(x) = \sigma(x) (1 - \sigma(x)). \quad (28)$$

Figure

Figure 2 shows the Sigmoid activation function and its derivative. Chain rule has been used to find the derivative.

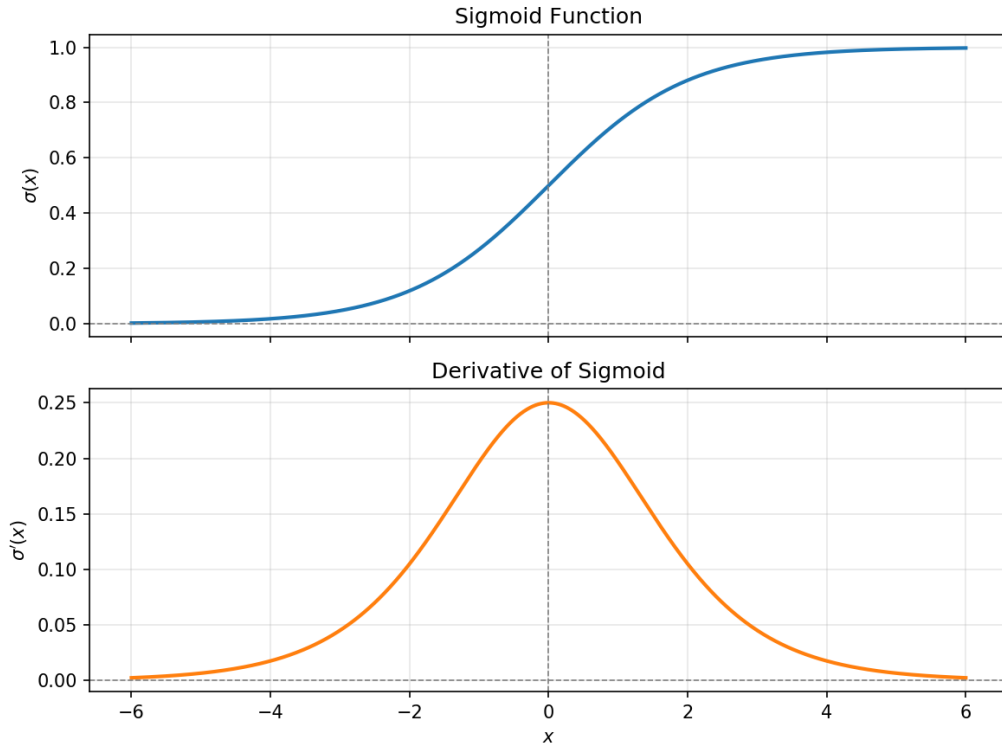


Figure 12: Sigmoid activation (top) and its derivative (bottom).

Python Implementation

```
1 import numpy as np
2
3 def sigmoid(x):
4     return 1.0 / (1.0 + np.exp(-x))
5
6 def sigmoid_derivative(x):
7     s = sigmoid(x)
8     return s * (1.0 - s)
```

Listing 4: Sigmoid and its derivative

(c) Hyperbolic Tangent

Analytical Form

The hyperbolic tangent is

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh(x)}{\cosh(x)}. \quad (29)$$

Its derivative follows from the quotient rule (or the identity $\frac{d}{dx} \tanh x = \text{sech}^2 x$):

$$\tanh'(x) = 1 - \tanh^2(x) = \text{sech}^2(x). \quad (30)$$

Figure

Figure 3 shows the Hyperbolic Tangent activation function and its derivative. Chain rule has been used to find the derivative.

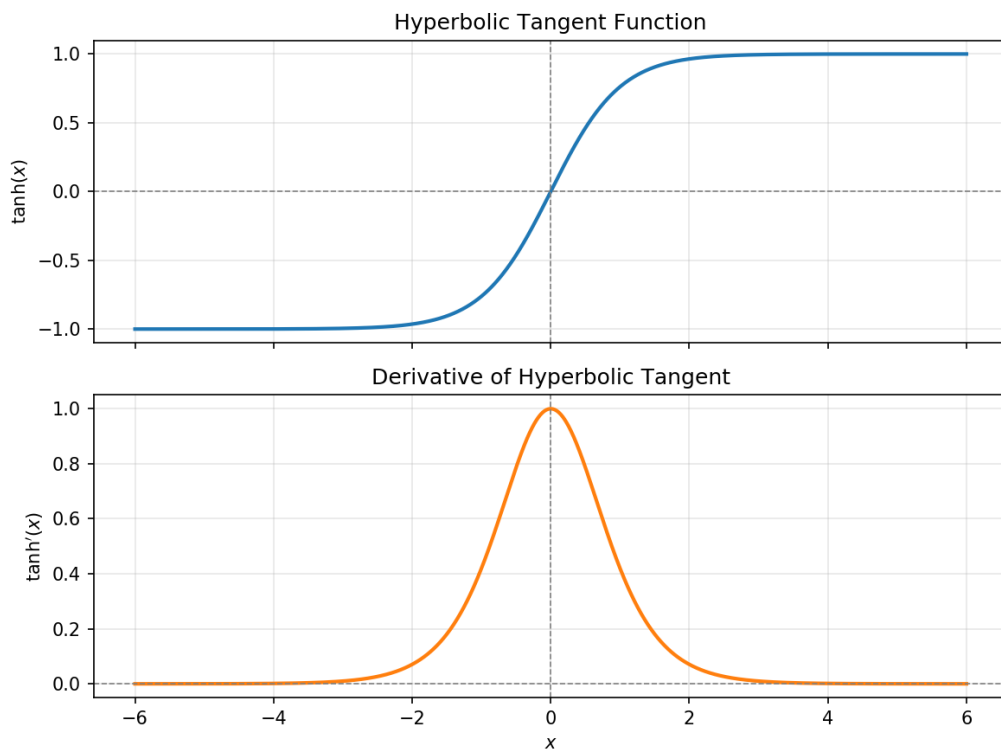


Figure 13: Hyperbolic tangent activation (top) and its derivative (bottom).

Python Implementation

```
1 import numpy as np
2
3 def tanh_fn(x):
4     return np.tanh(x)
5
6 def tanh_derivative(x):
7     t = np.tanh(x)
8     return 1.0 - t**2
```

Listing 5: Hyperbolic tangent and its derivative

References

- [1] Q. Peter He, *Information Entropy based Indices for Variable Selection Performance Assessment*, in Mario R. Eden, Marianthi G. Ierapetritou, Gavin P. Towler (Eds.), *Computer Aided Chemical Engineering*, Elsevier, Volume 44, 2018, pp. 2281–2286, ISSN 1570-7946, ISBN 9780444642417, <https://doi.org/10.1016/B978-0-444-64241-7.50375-X>.
- [2] Z. Khan, *Deep Learning Course* (source code and scripts), GitHub repository, https://github.com/ZawarKhan97/Deep_Learning_Course.
- [3] R. E. Blahut, *Information Theory and Coding*, in Wendy M. Middleton, Mac E. Van Valkenburg (Eds.), *Reference Data for Engineers* (Ninth Edition), Newnes, 2002, pp. 25-1–25-31, ISBN 9780750672917, <https://doi.org/10.1016/B978-075067291-7/50027-3>.